

Seu Cantinho - Relatório

Heloísa Dias Viotto

Guilherme Eduardo G. da Silva

1 de dezembro de 2025

1 Introdução

Este documento tem como finalidade ser um relatório para o trabalho da disciplina Design de Software lecionada pela Professora Simone Dominico.

- Camada de Entidades: definição das entidades do projeto.
- Camada de Repositório: armazenamento dos dados da aplicação.

2 Estilo Arquitetural

O estilo arquitetural definida é "Em camadas" em conjunto com "Hexagonal", devido a sua vantagens para o trabalho.

2.1 Arquitetura em camadas

A vantagem que mais se destaca no estilo em camadas é o isolamento e a escalabilidade, solicitado pelo enunciado do trabalho. Além disso, essa arquitetura destaca-se por facilitar a manutenção e o desenvolvimento modular do sistema, além de possuir um baixo acoplamento.

Cada camada possui a sua responsabilidade, sendo independentes uns dos outros, cada um é responsável por uma função, sendo a comunicação realizada entre os componentes do sistema segue um fluxo hierárquico e controlado.

Para a modelagem do problema, foi decidido usar a divisão em 5 camadas, sendo elas:

- Camada de Apresentação: seria responsável pela interação com o usuário.
- Camada de Controlador: realiza a ligação entre a interface a a lógica de negócios.
- Camada de Serviço: implementação da lógica de negócios.

2.2 Arquitetura Hexagonal

Para auxiliar na modelagem do projeto, foi considerado utilizar conceitos também da arquitetura hexagonal com o intuito de ajudar na visualização da aplicação como um todo. Mesmo não sendo totalmente fiel à arquitetura em si, é possível identificar no diagrama de componentes (Figura 2), a separação entre adaptadores (Camada de controle e repositórios), portas (Camada de serviço) e o domínio (Camada de entidade).

2.3 Linguagem e comunicação entre as camadas - *REST*

Para auxiliar nessa tarefa, foi definida a linguagem de programação *JavaScript*, executada no ambiente de execução *Node.js*. Além disso, adotou-se como padrão de comunicação o estilo arquitetural de comunicação "REST", que possibilita a interação entre os módulos da aplicação por meio de interfaces HTTP de forma simples, padronizada e independente de plataforma.

O *JavaScript* mostra-se particularmente adequado para o desenvolvimento de APIs REST, devido ao suporte nativo ao formato JSON e à integração facilitada entre as camadas de *backend* e *frontend*.

No que se refere ao *frontend*, foi definido o uso do *framework React Next.js*, que permite a criação de sites estáticos otimizados e aplicações dinâmicas, além

de possibilitar a renderização de páginas no lado do servidor (SSR). Essa abordagem melhora o desempenho, reduz o tempo de carregamento no cliente e facilita a integração direta com a API desenvolvida em *Node.js*. O *Next.js* também realiza automaticamente a compilação e construção do projeto, fornecendo atualizações instantâneas sem a necessidade de configurações adicionais.

Sobre a questão de persistência dos dados, foi decidido usar arquivos *JSON* como armazenamento das informações (*Repositórios*).

3 Detalhes de Implementação

Visando uma solução mais robusta do “Seu Cantinho”, foram definidos algumas implementações que mantêm o sistema coerente, evitando problemas de condições de corrida entre espaços para serem reservados, perda de informações e um baixo tempo nas operações do sistema.

Para evitar o *double-booking* seria implementado uma checagem de disponibilidade no momento do envio da requisição para o arquivo JSON, ou seja, será realizado a comparação do tempo após o envio da requisição, caso haja algum outro usuário que tenha feito uma reserva no mesmo local selecionado, será retornado uma mensagem de aviso.

Pensando em manter a consistência e a independência dos dados, cada módulo da aplicação possui seu próprio arquivo JSON, garantindo isolamento e evitando acoplamentos indesejados entre os serviços. Para intermediar o acesso a esses dados, foi adotado o padrão de projeto *Repository*, responsável por encapsular as operações de persistência e abstrair a comunicação entre o serviço e o banco de dados.

4 Diagramas UML

A seguir, encontram-se os diagramas UML de classe e componentes, sendo, respectivamente, as Figuras 1 e 2.

5 Decisões de Design

Logo no final do trabalho, foi lido novamente o enunciado do trabalho e ficamos em dúvida em quem seria o usuário do sistema desenvolvido: a Dona Maria como usuária única ou os próprios clientes. Como esta dúvida surgiu no final do trabalho, resolvemos manter apenas a Dona Maria como usuária do sistema, sendo dispensado a tela de login.

Pensando em expansão do sistema, resolvemos deixar a Filial como uma CRUD com uma tela particular. A conexão de outras entidades, como a reserva e o espaço, são feitas através da ID da Filial.

Devido a falta de APIs externas de pagamento, resolvemos retirar a função de estornar, sendo a remoção de uma reserva feita apenas retirando ela do repositório. O estorno do pagamento seria feito pela Dona Maria separadamente.

Para todas as tarefas de CRUD, deve-se ler o arquivo JSON inteiro. Dependendo do tamanho do sistema, pode causar lentidão. Poderia ser mais eficiente tendo uma variável no sistema que seria atualizada toda vez que houvesse uma escrita no arquivo, no entanto foi optado por não ser dessa forma devido a memória que seria gasta.

6 Trade-offs

6.1 Pontos Positivos

- Frontend simples e intuitivo;
- O *double-booking* é evitado;
- O cadastro de novas entidades possui todas as informações como obrigatórias, impedindo a criação da entidade sem que essas informações estejam preenchidas. Isso confere ao nosso sistema a característica de forte consistência e integridade dos dados.

6.2 Pontos Negativos

- O estorno deve ser feito separadamente da remoção da reserva, não havendo uma opção de cancelar a reserva, apenas removê-la;

Figura 1: Diagrama UML de Classe.

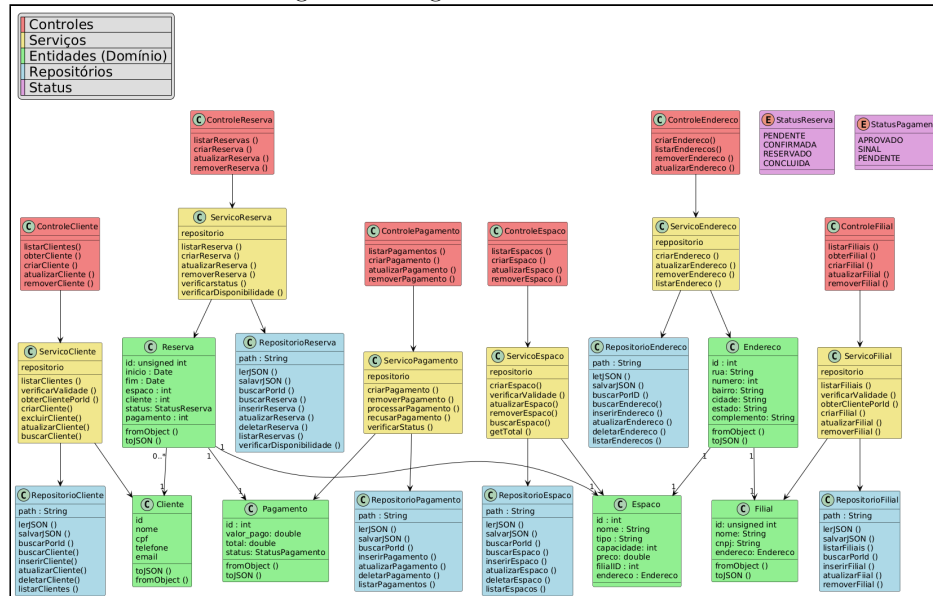
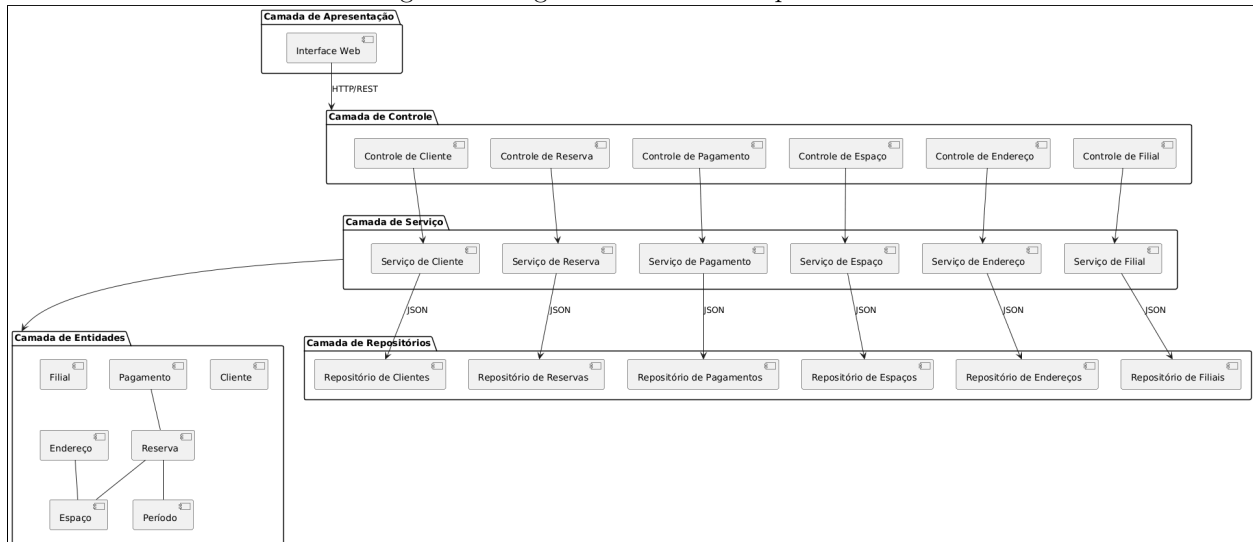


Figura 2: Diagrama UML de Componentes.



- O sistema foi pensado em um usuário único (Dona Maria), sendo este responsável por cadastrar todas as informações necessárias do sistema.
- Possível lentidão do sistema se houver muitas entidades gravadas.

7 Instruções de Execução do Sistema

Para iniciar, deve-se executar **pnpm i** no root para criar os arquivos necessários.

Para executar o backend, deve-se executar **node server.js** no diretório contendo o arquivo *server.js* em `src/backend/src`. Já para executar o frontend, deve-se, em outro terminal, executar **pnpm run dev** no diretório `src/frontend/src`.

Ambas as frentes executam em `localhost`, sendo o backend na porta 3000 e o frontend 3001. Portanto, para acessar ao sistema, deve-se entrar em **http://**

localhost:3001/.

Ao entrar, dona Maria verá uma mensagem de bem-vinda, juntamente com a logo do sistema e as opções de cadastro e visualização de entidades (clientes, filiais, endereços, reservas e espaços) no canto superior direito.

Em cada uma das entidade têm duas opções de interação com o sistema: uma para cadastro da entidade selecionada e outra com a listagem das entidades existentes. Na listagem normalmente se encontra os seguintes ícones: um X vermelho para remover a entidade; um ícone azul para atualizar os dados da entidade; um ícone de aviso sobre o *status* da reserva.

Na reserva, há um quarto botão, o qual abre um modal para definir uma quantidade a ser paga. Inicialmente, a reserva é criada com o status pendente sem ter sido realizado nenhum pagamento. Se o valor pago for maior que o preço a ser pago, não será feito. Se um valor for pago, mas não inteiro, será marcado o pagamento como sinal. Por fim, se for pago o valor completo, será marcado como aprovado.